AIC · AI+开源 · 答辩

SpringGuard

基于 Open Code Review 的 Spring 智能审查增强。
确定性规则 × 可选 AI × 可复现评测。

方向 AI 开发工具与开源协作

定位 领域增强工具 · 非薄封装部署

演示保底 无需 API Key 即可跑通

形态

Web · CLI · 规则包 · 评测

验证

自建金标准 6/6
干净样例 0 误报

仓库

NEX-coding/springguard

Spring 工程里，这些缺陷反复漏检

01

事务失效

`@Transactional` 同类自调用；异常吞掉破坏回滚

- 代理未生效导致“以为在事务里”
- 故障难复现、评审易漏

02

入参未校验

`@RequestBody` 缺 `@Valid`；可空链路直接 NPE

- 接口层缺少约束
- 运行期才爆，定位成本高

03

注入风险

MyBatis 注解误用 `${}`，进入评审盲区

- 字符串拼接 SQL
- 安全问题常被当成“能跑就行”

04

工具缺口

纯大模型不稳难复核；纯人工成本高

- 需要可演示、可复现的审查工具
- 目标：学习实训与中小团队后端

一条能当场跑通的审查流水线

INPUT

工程输入

内置样例或上传 Spring/Java ZIP（≤50MB）

- spring-buggy：六类已知缺陷
- spring-clean：干净负例对照
- 真实工程 ZIP 上传扫描

```
自动定位 src/main/java
```

CORE

双路径检测

确定性规则为主，OCR / 本地模型可选

- SpringGuard 静态规则引擎
- 项目级 .opencodereview/rule.json
- 结果字段统一后 merge

```
演示默认：仅 Guard，秒级出结果
```

OUTPUT

可核验输出

页面证据 + 金标准评测指标

- Web :5193 严重度筛选
- Recall / Precision 脚本复现
- 文件、行号、代码证据

```
python3 eval/run_eval.py
```

底座开源，增强自研，边界清楚

UPSTREAM

Open Code Review

alibaba/open-code-review

许可证：Apache-2.0

未修改上游源码

可选接入本地 Ollama

扩展走官方规则包与进程外编排

作用：提供通用 AI 审查入口；本轮模型增益未单独量化，不夸大。

TEAM BUILT

SpringGuard 层

确定性规则引擎

项目级 Spring 规则包

正负例 + 金标准评测

Web 上传与证据展示

资源清单与开放文档

价值：可定位、可复核、可复现；不是换皮部署上游 CLI。

团队实际交付了完整闭环

01

规则引擎

`scripts/spring_guard.py`

- Controller / Service / Mapper 规则
- 缺 @Valid、事务自调用、${} 等
- 秒级可复现，无模型依赖

02

Web 服务

`app_server.py + web/`

- 样例扫描 / ZIP 上传（≤50MB）
- 严重度筛选与证据代码
- 一键跑金标准评测

03

评测闭环

`eval/ · samples/`

- gold.json 六类缺陷条目
- 正例测召回，clean 测误报
- results.json 可脚本复核

另含规则包、编排脚本、演示视频与技术报告 / 资源清单

可脚本复现的验证结果

1.0
召回率 Recall
自建金标准 6 / 6

1.0
精确率 Precision
脚本口径 · 含 clean

0
干净样例误报
spring-clean

集合 / 模式	结果
spring-buggy + Guard	TP 6/6 · Recall 1.0
spring-clean	0 findings · 无误报
OCR / Hybrid	可选语义增强；本轮演示以 Guard 为主

如何理解指标

- 限于自建样例集，非泛化承诺
- 命中按规则/文本匹配口径
- 复现：python3 eval/run_eval.py

不是换皮部署，是领域增强

常见做法

薄封装风险

直接跑通用审查 CLI

结果难复核、难复现

Spring 高发点靠临场发挥

缺少正负例对照

开源边界与清单含糊

本作品

可验证改进

Spring 确定性规则可重复命中

证据定位到文件与行号

金标准 + 干净样例双验证

上游未改源码，扩展路径清晰

清单披露许可与自研边界

评审可直接核验与复现

01 公开仓库

代码 · 规则包 · 样例 · 评测 · Web · 文档

02 一键演示

`python3 scripts/app_server.py` → http://127.0.0.1:5193/

03 资源清单

区分上游 / 自研 / 可选模型，权利边界清晰

04 提交材料

技术报告、演示视频、答辩 PPT、佐证材料齐备

访问入口

材料以报名系统上传版本为准；仓库便于复核代码与评测脚本。

README 含安装、演示与评测步骤。

github.com/NEX-coding/springguard

验证更严，增强更深

01

评测加强

把“能复现”推进到“更可信”

- 行级匹配更严格
- 过滤注释干扰
- 扩展真实工程负例
- 增加人工抽检记录

阶段产出

更严的评测脚本 + 误报/漏报分析表

02

模型对照

把可选 AI 路径补成可比较实验

- 补全 OCR 对比实验
- 量化语义审查增益
- 记录超时 / 漏报案例
- 明确何时启用模型分支

阶段产出

Guard vs OCR 对照报告（含失败案例）

03

生态维护

从作品演示走向可持续维护

- 补充更多 Spring 规则
- CI 接入回归评测
- 完善仓库 LICENSE
- 视情况向上游贡献示例

阶段产出

CI 门禁 + 明确开源许可与贡献说明

NOW 自建样例可复现

CI / 许可 / 上游贡献

NEXT 更严评测 + 模型对照

THEN

THANKS

SpringGuard

把开源 AI 审查底座，变成可复现的 Spring 工程安全网。

github.com/NEX-coding/springguard